

Tools & Concepts Guide

Support Ticket Management System - why each tool/concept is used, pros & cons, and alternatives

Tools & Concepts Guide

Project: Support Ticket Management System

Purpose: Understand every major tool and concept used in this project - why it was chosen, pros & cons, and practical alternatives.

How to Use This Document

For each item you will find:

1. What it is - short definition
 2. Why we used it - role in this project
 3. Pros - strengths
 4. Cons - trade-offs / limitations
 5. Alternatives - other options you could use instead
-

Part A - Concepts (Architecture & Engineering Ideas)

1. Full-Stack Application

What it is

An application with both a user-facing frontend and a server-side backend, usually backed by a database.

Why we used it

The assessment requires a frontend, REST API, and database persistence working together.

Pros	Cons
End-to-end ownership of a feature	More moving parts than frontend-only work
Realistic engineering practice	Needs coordinated setup (env, DB, ports)
Easy to demo full user flows	Debugging can span UI + API + DB

Alternatives: Backend-only API + Postman; frontend + mock API (MSW); BaaS (Firebase/Supabase) without custom API.

2. Monorepo (npm Workspaces)

What it is

One git repository containing multiple packages/apps (here: apps/api and apps/web) managed from a single root.

Why we used it

Keeps API and web app versioned together with shared root scripts (npm run dev, npm run test).

Pros	Cons
One clone, one place for docs/artifacts	Can get large as projects grow
Shared tooling and lockfile	Needs clear package boundaries
Simple local orchestration	CI can be more complex at scale

Alternatives: Separate repos for api/web; pnpm/yarn workspaces; Nx; Turborepo; Lerna.

3. REST API

What it is

HTTP-based API style using resources and verbs (GET, POST, PATCH) and JSON payloads.

Why we used it

Clean, well-understood contract between React and Express for tickets, users, and comments.

Pros	Cons
Easy to test with curl/Postman	Can over-fetch or under-fetch data
Browser/network tools show every call	Versioning needs discipline
Language-agnostic clients	Not ideal for highly nested graph queries

Alternatives: GraphQL; tRPC; gRPC; JSON-RPC; Server Actions (Next.js).

4. State Machine (Ticket Status Lifecycle)

What it is

A model of allowed transitions between states. From a given status, only certain next statuses are valid.

Why we used it

Ticket status (OPEN -> IN_PROGRESS -> RESOLVED -> CLOSED, plus cancel paths) is the Core judgment piece - must be enforced on the backend.

Pros	Cons
Business rules become explicit	Extra code vs free-form status field
Easy to test valid/invalid paths	Must keep UI map aligned with backend
Prevents illegal lifecycle jumps	Complex domains need richer libraries

Alternatives: XState; workflow engines (Temporal); DB check constraints; validation in service only without dedicated module.

5. ORM (Object-Relational Mapping)

What it is

A layer that maps database tables to code objects/models and generates SQL for you.

Why we used it

Prisma provides typed models for User, Ticket, Comment plus migrations and seeding.

Pros	Cons
Less raw SQL for common CRUD	Advanced SQL can feel restricted
Schema as source of truth	Learning curve for generated client
Safer refactors with types	Extra abstraction over the database

Alternatives: Drizzle; TypeORM; Sequelize; Knex (query builder); raw pg SQL.

6. Database Migrations

What it is

Versioned SQL (or schema diffs) that evolve the database structure safely over time.

Why we used it

prisma migrate creates repeatable schema setup so any machine can recreate the DB.

Pros	Cons
Reproducible environments	Bad migrations are hard to undo in production
History of schema changes	Teams must coordinate migration ownership
Works for CI and new developers	Conflicts if two people change schema differently

Alternatives: Flyway; Liquibase; Knex migrations; plain SQL scripts checked into git.

7. Seed Data

What it is

Sample records inserted so the app is usable immediately after setup.

Why we used it

Assessment requires seeded users/tickets so reviewers can run the app without manual data entry.

Pros	Cons
Instant demo-ready UI	Seeds can overwrite local experimentation
Consistent demos for reviews	Must keep seed script updated with schema
Good for onboarding	Not a substitute for real fixtures in all tests

Alternatives: Factory libraries (fishery); SQL dump restore; Docker init scripts.

8. Input Validation

What it is

Checking request/form data before trusting it (required fields, enums, formats).

Why we used it

Backend Zod schemas reject invalid tickets/comments; frontend Zod + React Hook Form give early UX feedback.

Pros	Cons
Fewer bad records in DB	Must validate on server even if UI validates
Clear field-level errors	Duplicate schemas FE/BE unless shared package
Types can be inferred from schemas	Over-strict rules can frustrate users

Alternatives: Joi; Yup; AJV (JSON Schema); class-validator; express-validator.

9. Integration Testing

What it is

Tests that exercise multiple layers together (HTTP -> service -> DB), not just one unit function.

Why we used it

Mandatory assessment test tier: prove status transitions succeed/fail via the real API.

Pros	Cons
High confidence in critical rules	Slower than unit tests
Catches wiring bugs	Needs a running database
Matches real API contract	Harder to isolate failures

Alternatives: Unit tests only; E2E (Playwright/Cypress); contract tests (Pact).

10. Environment Variables

What it is

Config values (DATABASE_URL, PORT) loaded from .env, not hard-coded.

Why we used it

Different machines need different secrets/hosts; assessment forbids committing secrets.

Pros	Cons
Secrets stay out of git	Easy to forget a required variable
Same code, different environments	Typo can break runtime
Standard 12-factor practice	Local/prod drift if not documented

Alternatives: Doppler; Vault; AWS Secrets Manager; .env with dotenv only (we rely on Prisma/Node env loading).

11. Containerized Database (Docker Compose)

What it is

Run PostgreSQL in a container defined by docker-compose.yml instead of installing Postgres natively.

Why we used it

One command (`npm run db:up`) gives every developer the same DB version and credentials.

Pros	Cons
Consistent local DB	Requires Docker Desktop
Easy tear-down/reset	Port conflicts on 5432
Close to production Postgres	Slight resource overhead

Alternatives: Local Homebrew Postgres; cloud Postgres (Neon/Supabase); SQLite for local-only apps.

12. Client-Server Separation

What it is

UI (React) and API (Express) are separate processes communicating over HTTP.

Why we used it

Clear ownership of concerns; matches common production architecture.

Pros	Cons
Independent scaling/deploy	CORS and proxy config needed
Different teams can own layers	Two servers to run in development
Easy to add mobile clients later	Network latency vs server-rendered apps

Alternatives: Next.js full-stack; Remix; monolithic server-rendered Express + EJS.

Part B - Frontend Tools

13. React 18

What it is

A JavaScript library for building component-based UIs.

Why we used it

Assessment/stack preference; strong ecosystem for forms, routing, and data fetching.

Pros	Cons
Huge ecosystem and hiring pool	Easy to over-complicate state
Component model fits CRUD UIs	Requires build tooling
Excellent learning resources	Not a full framework by itself

Alternatives: Vue; Svelte; Angular; Solid; Preact.

14. TypeScript

What it is

JavaScript with static types compiled to JS.

Why we used it

Catch ticket/status/shape mistakes at compile time on both API and web.

Pros	Cons
Safer refactors	Slightly more ceremony
Better editor autocomplete	Build step required
Shared mental model with Prisma types	Learning curve for generics

Alternatives: Plain JavaScript; Flow (legacy); JSDoc typed JS.

15. Vite

What it is

Modern frontend build tool and dev server with very fast Hot Module Replacement (HMR).

Why we used it

Fast React + TypeScript DX; also proxies /api to the Express server in development.

Pros	Cons
Extremely fast local reloads	Config differs from Webpack/CRA
Simple project setup	Ecosystem plugins fewer than Webpack historically
First-class TS/React support	Must understand proxy for API calls

Alternatives: Create React App (legacy); Next.js; Parcel; Webpack; Rsbuild.

16. React Router

What it is

Client-side routing library for React SPAs.

Why we used it

Navigate between list (/), create (/tickets/new), and detail (/tickets/:id) without full page reloads.

Pros	Cons
Mature and widely used	SPA routing vs server routes trade-offs
Nested layouts (Layout outlet)	Loaders/actions model differs from Next
Easy param reading (useParams)	Deep linking needs correct server fallback in prod

Alternatives: TanStack Router; Next.js App Router; Wouter; Reach Router (legacy).

17. TanStack Query (React Query)

What it is

Library for fetching, caching, and synchronizing server state in React.

Why we used it

Ticket lists/details come from the API; mutations invalidate caches so UI stays fresh.

Pros	Cons
Built-in loading/error/caching	Extra concept beyond useEffect
Mutation + invalidation patterns	Overkill for tiny static pages
Dedupes requests	Need good queryKey design

Alternatives: SWR; RTK Query; Apollo (GraphQL); plain fetch + useEffect; Zustand with manual fetch.

18. React Hook Form

What it is

Performant form state library built around uncontrolled inputs and hooks.

Why we used it

Create/edit ticket and comment forms with validation errors and low re-render cost.

Pros	Cons
Excellent performance	Mentally different from fully controlled forms
Works great with Zod resolvers	Some complex UIs need Controllers
Small API for common forms	Learning curve for advanced fields

Alternatives: Formik; Final Form; native controlled React state; Conform.

19. Zod (Frontend + Backend)

What it is

TypeScript-first schema validation library.

Why we used it

Validate API bodies and frontend forms with the same style of schemas.

Pros	Cons
Schemas -> TypeScript types	Bundle size on frontend if overused
Great error messages	Duplication if FE/BE not shared
Works server and client	Advanced transforms can get dense

Alternatives: Yup; Joi; Valibot; Superstruct; TypeBox.

20. Tailwind CSS

What it is

Utility-first CSS framework (classes like flex, p-4, text-sm).

Why we used it

Fast UI styling for list/detail layouts without large custom CSS files.

Pros	Cons
Rapid UI iteration	Class-heavy JSX
Consistent spacing/colors	Design can look generic without tokens
Easy responsive utilities	Team discipline needed for reuse

Alternatives: CSS Modules; styled-components; Emotion; Bootstrap; MUI styling; vanilla CSS.

21. Radix UI Primitives (+ shadcn-style components)

What it is

Unstyled, accessible component primitives (Select, Label, Toast, Slot). Styled with Tailwind in this repo.

Why we used it

Accessible dropdowns/toasts without importing a heavy UI kit's design language.

Pros	Cons
Accessibility baked in	More setup than a full kit
Full style control	You maintain the component copies
Composable primitives	Learning Radix APIs

Alternatives: MUI; Chakra UI; Ant Design; Headless UI; React Aria Components.

22. Lucide React

What it is

Icon pack for React (Search, ArrowLeft, Ticket, etc.).

Why we used it

Lightweight, consistent icons in the header and actions.

Pros	Cons
Clean default style	Another dependency
Tree-shakeable icons	Icon naming differs across packs
Easy to swap icons	Not a design system by itself

Alternatives: Heroicons; Font Awesome; Material Icons; Phosphor; react-icons.

23. clsx + tailwind-merge + CVA

What they are

Helpers for building class names: conditional classes (clsx), conflict resolution (tailwind-merge), variant APIs (class-variance-authority).

Why we used them

Clean Button/Badge variants without messy string concatenation.

Pros	Cons
------	------

Predictable component APIs	Small stack of micro-deps
Avoids conflicting Tailwind classes	Overkill for a couple of buttons
Matches shadcn patterns	Need team convention

Alternatives: Manual template strings; classnames package; Stitches variants; Vanilla Extract.

Part C - Backend Tools

24. Node.js

What it is

JavaScript runtime for server-side code.

Why we used it

Same language as React, strong ecosystem for Express/Prisma, assessment-friendly stack.

Pros	Cons
One language across stack	CPU-heavy work needs care (event loop)
Huge package ecosystem	Type safety needs TypeScript
Easy local scripting	Packaging/deploy models vary

Alternatives: Java/Spring; Python/FastAPI/Django; Go/Gin; .NET; Ruby on Rails.

25. Express

What it is

Minimal Node.js web framework for HTTP routes and middleware.

Why we used it

Simple REST surface for tickets/users with custom state-machine logic in services.

Pros	Cons
Tiny and flexible	Less structure than NestJS
Huge middleware ecosystem	Easy to grow inconsistently
Easy for beginners and reviews	Manual project organization

Alternatives: Fastify; NestJS; Koa; Hono; Next.js route handlers.

26. Prisma

What it is

Next-generation ORM with schema file, migrations, generated client, and Studio UI.

Why we used it

Fast path to PostgreSQL models, migrations, seed, and typed queries.

Pros	Cons
Excellent TypeScript DX	Complex joins/raw SQL sometimes awkward
Migrations + Studio included	Generated client must be regenerated
Clear schema readability	Some teams prefer lighter query builders

Alternatives: Drizzle; TypeORM; Sequelize; MikroORM; Kysely.

27. PostgreSQL

What it is

Reliable open-source relational database.

Why we used it

Assessment-approved; strong for relational data (users ? tickets ? comments) and enums.

Pros	Cons
ACID, mature, powerful SQL	Heavier than SQLite for tiny demos
Excellent tooling	Needs process/container management
Great for production paths	Schema changes need migrations

Alternatives: MySQL/MariaDB; SQLite; MongoDB; CockroachDB; Neon (serverless Postgres).

28. CORS

What it is

Middleware that allows browsers on another origin (e.g. :5173) to call the API (:3001).

Why we used it

Frontend and backend run on different ports in development.

Pros	Cons
Solves browser same-origin limits	Misconfig can expose APIs broadly
Simple to enable for local demos	Not a substitute for auth
Works with credentials carefully configured	Easy to forget in production hardening

Alternatives: Same-origin via reverse proxy/Nginx; Vite proxy only (dev); gateway layer.

29. tsx

What it is

Tool to run TypeScript files directly (with watch mode) without a separate compile step in development.

Why we used it

npm run dev:api uses tsx watch src/index.ts for fast backend reloads.

Pros	Cons
Instant TS execution	Not the same as production node dist
Great DX for APIs	Watch process can leave ports occupied
Simple scripts (seed too)	Need real tsc build for prod

Alternatives: ts-node; nodemon + tsc -w; Bun; compiled JS with node --watch.

Part D - Testing & Tooling

30. Vitest

What it is

Fast unit/integration test runner with Vite-compatible config, Jest-like API.

Why we used it

Run API integration tests for status transitions quickly.

Pros	Cons
Fast, modern, TS-friendly	Ecosystem still evolving vs Jest in some niches
Simple config	Need setup discipline for DB tests
Works well in this monorepo	Browser component testing differs from RTL setups

Alternatives: Jest; Mocha; node:test; AVA.

31. Supertest

What it is

HTTP assertion library that hits an Express app without binding a real port (or against a running server).

Why we used it

request(app).patch('/api/tickets/:id/status') verifies 200 vs 422 for transitions.

Pros	Cons
Excellent for REST APIs	Doesn't replace browser E2E
Clean status/body assertions	Still needs DB for integration depth
Plays well with Vitest/Jest	Async DB cleanup must be careful

Alternatives: light-my-request (Fastify); Playwright API testing; curl scripts in CI.

32. concurrently

What it is

Run multiple npm scripts in one terminal (API + web together).

Why we used it

npm run dev starts both servers for one-command local development.

Pros	Cons
Convenient single command	Log noise from two processes
Cross-platform	Harder to restart one side only (use dev:api/dev:web)
Simple config	Not a process supervisor for production

Alternatives: npm-run-all; terminal multipane (tmux); Docker Compose for all services; Turborepo dev.

33. Docker / Docker Compose

What it is

Container runtime + YAML orchestration for services (Postgres here).

Why we used it

Reproducible local database matching production-like Postgres.

Pros	Cons
Identical DB across machines	Disk/CPU cost; Docker must be running
Simple up/down lifecycle	Volume issues if paths wrong
Easy reset of environment	Networking quirks for beginners

Alternatives: Podman Compose; local DB install; cloud managed DB for all environments.

34. Git + GitHub

What it is

Version control (Git) hosted on GitHub for collaboration and submission.

Why we used it

Assessment submission expects an accessible repository and meaningful commit history.

Pros	Cons
Standard industry practice	History hygiene matters for reviews
PR workflow for changes	Private/public access must be set correctly
Traceability of AI corrections via commits	Secrets accidentally committed can be catastrophic

Alternatives: GitLab; Bitbucket; Azure Repos; self-hosted Gitea.

35. Cursor (AI Coding Tool)

What it is

AI-assisted IDE used for requirements breakdown, generation, debugging, and docs.

Why we used it

Primary approved tool for this AI capability exercise; workflow artifacts live under tool-specific/cursor-workflow/.

Pros	Cons
Speeds scaffolding and iteration	Can produce shallow/wrong code if not reviewed
Great for docs/tests/boilerplate	Prompt history must be curated for evidence
Spec-driven workflows work well	Ownership still required by the engineer

Alternatives: Claude Code / Claude chat; GitHub Copilot; Kiro; JetBrains AI; ChatGPT with careful copy of context.

Part E - Cross-Cutting Patterns Used in This Repo

36. Structured API Errors

Pattern: Responses like { message, errors: [{ field, message }] }.

Why: Frontend can show toast/field errors consistently for 400/422 cases.

Alternatives: Problem Details (RFC 7807); GraphQL errors array; ad-hoc string messages.

37. Service Layer

Pattern: Routes stay thin; business logic lives in ticket.service.ts.

Why: Easier testing and clearer separation of HTTP vs domain rules.

Alternatives: Fat controllers; hexagonal/clean architecture folders; NestJS modules/services.

38. Seeded Users Without Auth (Core Scope)

Pattern: No login; default seeded user creates tickets/comments.

Why: Assessment Core focuses on lifecycle evidence, not authentication.

Alternatives (Stretch): JWT sessions; NextAuth; Clerk; role-based protected routes.

Quick Comparison Tables

Frontend Framework Alternatives

Choice	When it shines	Trade-off
React (this project)	Ecosystem, hiring, flexibility	DIY architecture
Vue	Gentle learning curve	Smaller job market in some regions
Svelte	Less boilerplate	Smaller ecosystem
Angular	Large enterprise apps	Heavier learning curve
Next.js	SSR/SEO/full-stack	More opinionated deploy model

Backend Framework Alternatives

Choice	When it shines	Trade-off
Express (this project)	Simple REST APIs	Manual structure
Fastify	Performance + schema	Different plugin model
NestJS	Large structured apps	Boilerplate weight
FastAPI	Python data/ML teams	Different runtime
Spring Boot	Enterprise Java shops	Heavier setup

Database Alternatives

Choice	When it shines	Trade-off
PostgreSQL (this project)	Relational integrity	Needs service/container
SQLite	Tiny demos, zero ops	Weaker concurrent write story
MySQL	Common hosting support	Feature differences vs Postgres
MongoDB	Flexible documents	Relations harder for this domain

ORM Alternatives

Choice	When it shines	Trade-off
Prisma (this project)	TS DX + migrations	Binary/engine abstraction
Drizzle	SQL-like, lightweight	Younger ecosystem
TypeORM	Decorators, NestJS world	Inconsistent community opinions
Raw SQL	Full control	More boilerplate

Mental Model of This Stack

```
Browser (React + TanStack Query + Router)
  | HTTP JSON (REST)
Express (routes -> Zod -> services -> state machine)
  | Prisma Client
PostgreSQL (Docker)
```

Key rule of the project: UI can hide invalid status buttons, but only the backend state machine is the real enforcement. Tests prove that.

Part F - Middleware in This Project

39. What Is Middleware?

What it is

In Express, middleware is a function that runs during the request/response cycle. It receives (req, res, next), can read/modify the request, send a response, or call next() to pass control to the next middleware/route.

Request -> middleware 1 -> middleware 2 -> route handler -> error middleware -> Response

Typical jobs: parse JSON body, enable CORS, authenticate users, log requests, handle errors.

40. Middleware Used in This Project

Configured in apps/api/src/app.ts and apps/api/src/middleware/error-handler.ts.

Order of execution

1. cors()
2. express.json()
3. Route handlers (/health, /api/tickets, /api/users)
??? each route wrapped with asyncHandler(...)
4. errorHandler (last - 4-argument error middleware)

1) cors() - Cross-Origin Resource Sharing

Item	Detail
Package	cors
Where	app.use(cors()) in apps/api/src/app.ts
What it does	Allows the React app on http://localhost:5173 to call the API on http://localhost:3001 from the browser
Benefits	Without CORS, the browser blocks cross-origin fetch calls even if the API is running
Note	Dev Vite proxy can reduce CORS pain; CORS still helps when calling :3001 directly (or from Postman-like browser tools)

2) express.json() - JSON body parser

Item	Detail
Package	Built into Express
Where	app.use(express.json()) in apps/api/src/app.ts
What it does	Reads Content-Type: application/json bodies and puts parsed data on req.body
Benefits	Create/update ticket and comment endpoints can use req.body.title, req.body.status, etc.
Without it	req.body is undefined - Zod validation and create/update would fail

3) asyncHandler - async route wrapper

Item	Detail
File	apps/api/src/middleware/error-handler.ts
Where used	Wrapping every route in apps/api/src/routes/index.ts
What it does	Catches rejected Promises from async route handlers and forwards them to next(err)
Benefits	One place for errors; avoids unhandled promise rejections; keeps route code clean (try/catch not needed everywhere)

4) errorHandler - centralized error middleware

Item	Detail
File	apps/api/src/middleware/error-handler.ts
Where	app.use(errorHandler) registered last in app.ts
What it does	Converts known errors into consistent JSON: { message, errors: [{ field, message }] }
Handles	AppError (validation/business), ZodError (400), StatusTransitionError (422), Prisma not-found (404), unknown (500)
Benefits	Frontend always gets the same error shape; status codes map to real failure types; no duplicated error formatting in routes

Middleware not used (intentionally - Core scope)

Middleware	Why skipped
Auth (JWT/session)	Auth is Stretch/optional in the assessment
Rate limiting	Not required for Core demo
Request logging (morgan)	Could be added for production observability
Helmet (security headers)	Production hardening, not Core

Pros of current middleware stack: small, clear, easy to explain in reviews.

Cons: production apps usually add auth, logging, rate limits, and stricter CORS origins.

Alternatives people often add later: helmet, morgan/pino-http, express-rate-limit, custom authMiddleware, zod-express middleware wrappers.

Part G - Database Configuration & Switching to MySQL

41. Where Database Config Is Defined

Database settings are split across a few files (each has a role):

Location	What it controls
apps/api/prisma/schema.prisma -> datasource db	Provider (postgresql) and reads URL from env
apps/api/.env (local, not committed)	Actual DATABASE_URL secret/connection string
.env.example	Template for developers (safe to commit)
docker-compose.yml	Local PostgreSQL container (image, port, user, password, db name)
apps/api/src/lib/prisma.ts	Prisma Client singleton used by services
README.md / docs/DEVELOPER_GUIDE.md	Setup instructions for humans

Current Prisma datasource

```
// apps/api/prisma/schema.prisma
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}
```

Current connection string shape

```
# apps/api/.env (and .env.example)
DATABASE_URL=postgresql://postgres:postgres@localhost:5432/support_tickets
```

Format: postgresql://USER:PASSWORD@HOST:PORT/DATABASE

Docker Postgres defaults (docker-compose.yml)

Setting	Value
Image	postgres:16-alpine
Host port	5432
User	postgres
Password	postgres
Database	support_tickets

42. Changing Database from PostgreSQL to MySQL

Prisma supports MySQL. For this project, you mainly change provider, connection URL, Docker service, and regenerate migrations - not rewrite business logic.

Step-by-step changes

1) Update Prisma provider

In apps/api/prisma/schema.prisma:

```
datasource db {
  provider = "mysql"
  url      = env("DATABASE_URL")
}
```

2) Update connection string

In apps/api/.env and .env.example:

```
# MySQL style
DATABASE_URL=mysql://root:mysql@localhost:3306/support_tickets
```

Format: mysql://USER:PASSWORD@HOST:PORT/DATABASE

3) Replace Docker Compose service

Swap the Postgres service for MySQL, for example:

```
services:
  mysql:
    image: mysql:8.4
    container_name: support-ticket-db
    restart: unless-stopped
    ports:
      - "3306:3306"
    environment:
      MYSQL_ROOT_PASSWORD: mysql
      MYSQL_DATABASE: support_tickets
    volumes:
      - mysql_data:/var/lib/mysql

volumes:
  mysql_data:
```

Also update root scripts/docs that say db:up starts Postgres.

4) Recreate migrations (important)

PostgreSQL migration SQL is not portable as-is (enums/types differ).

Recommended approach for a project this size:

```
# Stop old DB, start MySQL
npm run db:down
# after docker-compose change:
docker compose up -d

cd apps/api
# Remove old Postgres migrations (or archive them), then:
rm -rf prisma/migrations
npx prisma migrate dev --name init_mysql
npx prisma generate
npm run db:seed
```

5) Regenerate Prisma Client and restart API

```
cd apps/api
npx prisma generate
npm run dev
```

6) Update docs / tests / SQL examples

Area	What to change
docs/DEVELOPER_GUIDE.md	Ports (3306), connection string, psql -> mysql client
Sample SQL	MySQL uses backticks often; less quoted tags than Postgres
Case-insensitive search	This app uses Prisma mode: 'insensitive' - supported on MySQL through Prisma, but collation/settings can matter
Enums	Prisma still models enums; MySQL maps them differently under the hood than Postgres

Schema features in this project that usually transfer fine

- String / DateTime / relations / @id cuid
- Prisma enums (Role, Priority, TicketStatus) - Prisma maps them for MySQL
- Seed script (prisma/seed.ts) - no Postgres-specific SQL

Things to double-check after switching

1. Case-insensitive search in ticket.service.ts (contains + mode: 'insensitive') still behaves as expected
2. Integration tests still pass: npm run test
3. Prisma Studio still opens: npx prisma studio

4. No leftover Postgres-only SQL in old migration folders

Effort estimate

Change type	Effort
Config + Docker + new migration	Low-medium
Business logic rewrite	Usually none (Prisma abstracts most of it)
Doc/SQL examples update	Low

Pros / cons of switching to MySQL

Pros	Cons
Familiar if your team already runs MySQL	Lose some Postgres features (advanced types/JSONB ergonomics)
Widely hosted on cheap shared hosting	Need new migrations; old Postgres SQL not reusable
Prisma makes provider switch relatively small	Collation/search behavior can differ

Alternatives instead of self-managed MySQL

Option	Notes
Keep PostgreSQL (recommended default)	Best fit for this project as built
PlanetScale / TiDB	MySQL-compatible hosted options
SQLite (provider = "sqlite")	Fastest local demo; weaker concurrency
MongoDB	Would require larger schema redesign (not a drop-in)

Suggested Learning Order

1. REST + HTTP status codes (200/400/404/422)
 2. React components + React Router
 3. Express routes + middleware
 4. Prisma schema + migrations + seed
 5. Zod validation
 6. TanStack Query mutations/invalidations
 7. State machine design + integration tests
 8. Docker Compose for local Postgres
-

Related Project Docs

Document	Focus
docs/DEVELOPER_GUIDE.md	Setup, debug, SQL, package cheat sheet
docs/design.md	Architecture decisions
docs/requirements.md	Feature requirements
docs/testing-notes.md	Test strategy
tool-workflow.md	How AI was used across the lifecycle
prompt-history/	Prompt chat evidence

End of document - Support Ticket Management System